

Лекция 9. Основы Docker и контейнеризация приложений

Цель лекции

Понять отличие контейнера от виртуальной машины
Разобрать базовый жизненный цикл image и container
Запустить Nginx в контейнере, опубликовать порт и
проверить приложение
Подключить данные через bind mount и named volume
Собрать собственный image и описать запуск через
Docker Compose

Учебные вопросы

1. Контейнер и виртуальная машина; Docker Client, Daemon и Registry
2. Image, Container и жизненный цикл
3. Запуск, остановка, restart, удаление и публикация портов
4. Bind Mount, Named Volume и Docker Network
5. Logs, Exec, Restart Policy и базовый Healthcheck
6. Dockerfile, Build и Docker Compose
7. Базовые правила безопасности Docker

Главный учебный сценарий

Pull Image → Run Container → Publish Port

Publish Port → Test Application → Logs / Exec

Logs / Exec → Bind Mount → Recreate Container

Recreate Container → Build Image → Compose

Compose → Diagnose

Не вводим много независимых сценариев: один путь проходит через все базовые навыки

Единый жизненный цикл Docker-приложения



Основные команды

- `docker pull`
- `docker run`
- `docker ps`
- `docker logs`
- `docker exec`
- `docker build`
- `docker compose up -d`



Главный вывод

Docker-приложение проходит цикл: image → запуск → контейнер → данные и порты → диагностика → обновление и пересоздание.

1. Container отличается от Virtual Machine

Container - изолированный процесс или группа процессов

Container использует Kernel Host

Virtual Machine имеет отдельную Guest OS

Virtual Machine имеет отдельный Kernel

Container обычно запускается быстрее и занимает меньше ресурсов

VM сильнее изолирует ОС, но тяжелее по ресурсам

Namespaces и Cgroups дают базовую изоляцию контейнера

Namespaces → изоляция процессов, сети, mount points и других объектов

Cgroups → ограничение и учет ресурсов

Контейнер не является полноценной виртуальной машиной

Контейнер все равно зависит от ядра Docker Host

Capabilities на этом этапе только называем, без глубокого разбора

Схема: виртуальная машина и контейнер

Два подхода к изоляции приложений

ВИРТУАЛЬНАЯ МАШИНА



КОНТЕЙНЕР



Быстрее запуск

Контейнеры запускаются за секунды, так как не загружают гостевую ОС



Меньше ресурсов

Контейнеры используют общее ядро хоста и требуют меньше RAM и CPU



Больше плотность

На одном хосте можно запустить больше контейнеров, чем виртуальных машин



Изоляция

VM: сильная изоляция на уровне гипервизора
Контейнер: изоляция на уровне ядра Linux



Запуск

VM: десятки секунд и больше
Контейнер: доли секунд



Ресурсы

VM: больше накладных расходов
Контейнер: меньше накладных расходов



Управление

VM: сложнее, больше компонентов
Контейнер: проще, меньше компонентов



Образ

VM: образ включает гостевую ОС
Контейнер: образ только с приложением



Важно

Контейнер ≠ VM
Контейнеры не заменяют виртуальные машины, а дополняют их



Итог:

Контейнер использует ядро хоста и обеспечивает легковесную изоляцию процессов, файловой системы, сети и ресурсов. Виртуальная машина содержит полноценную гостевую ОС и обеспечивает более сильную изоляцию на уровне гипервизора.

Архитектура Docker начинается с CLI и Daemon

Administrator выполняет команды docker CLI

docker CLI обращается к Docker Daemon

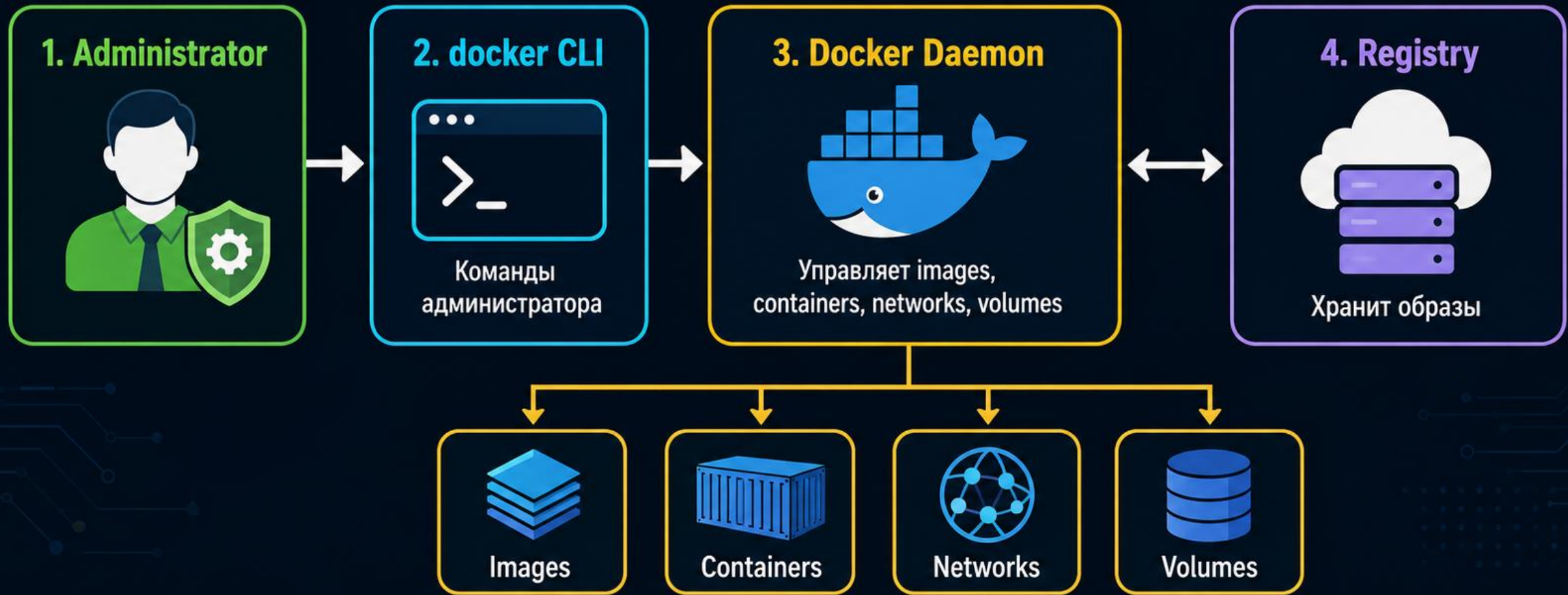
Docker Daemon управляет Images, Containers, Networks и Volumes

Docker Daemon получает images из Registry

Docker API и low-level Runtime упоминаем обзорно

Junior должен понимать, кто принимает команду и кто реально запускает container

Administrator, docker CLI, Docker Daemon и Registry



Главный вывод

Администратор работает через docker CLI, а Docker Daemon управляет контейнерами и получает образы из Registry.

Docker Socket дает очень высокие привилегии

`/var/run/docker.sock` - локальный socket управления Docker Daemon

Доступ к socket позволяет управлять контейнерами, image, network и volumes

Группа docker предоставляет практически административные возможности на Docker Host

Нельзя бездумно добавлять обычных пользователей в группу docker

Rootless Docker можно упомянуть как дополнительный вариант

2. Image - шаблон, Container - экземпляр

Image - шаблон с файловой системой и метаданными запуска

Container - запущенный или остановленный экземпляр Image

Один Image может использоваться для многих Containers

Container имеет собственное имя, состояние, сеть и writable layer

Удаление Image и удаление Container - разные действия

Writable Layer не подходит для важных данных

Container имеет изменяемый слой - Writable Layer
Файлы, созданные внутри Container без volume,
находятся в этом слое

docker stop не удаляет Writable Layer

docker start поднимает существующий Container с тем же Writable Layer

docker rm удаляет Container и его Writable Layer

Важные данные нельзя хранить только в Writable Layer

Тэг удобен, но не гарантирует неизменность image

Тэг - удобное имя image

Тэг может со временем указывать на другое содержимое

Version Tag обычно конкретнее, чем latest

Digest указывает на точное содержимое image

Для лаборатории используем согласованный tag
nginx:stable-alpine

Для строгой воспроизводимости используют digest

Жизненный цикл Container нужно различать по командам

`docker run` → создать новый container и запустить

`docker start` → запустить существующий container

`docker stop` → остановить существующий container

`docker restart` → перезапустить существующий container

`docker rm` → удалить container

Путаница `run` и `start` часто создает лишние containers

Типовая ошибка: после stop запускают второй container

Администратор остановил container: `docker stop web`

Затем снова выполнил `docker run ...`

В результате создается новый container, а не запускается старый

Правильно для старого container: `docker start web`

Проверка: `docker ps -a` показывает все containers, включая остановленные

Команды просмотра показывают состояние Docker

`docker ps` - показать запущенные containers

`docker ps -a` - показать все containers

`docker inspect web` - подробная информация о container

`docker image ls` - список images

`docker container prune` удаляет остановленные containers и требует осторожности

Container работает, пока работает главный процесс

Главный процесс внутри container выполняет роль PID 1

Если главный процесс завершился, container переходит в Exited

docker stop сначала просит процесс завершиться корректно

Затем при необходимости Docker завершает процесс принудительно

SIGTERM и SIGKILL можно знать в скобках, но глубокий signal handling не нужен

3. Первый запуск Nginx начинается с pull

`docker version` - проверить доступность Docker Client и Server

`docker pull nginx:stable-alpine` - скачать image из Registry

`docker image ls` - убедиться, что image появился локально

Pull Image - первый шаг практического сценария

Если pull не работает, проверяют сеть, DNS и доступ к registry

Безопасный первый запуск Nginx

```
docker run --name web -d -p 127.0.0.1:8080:80  
nginx:stable-alpine
```

--name web задает имя container

-d запускает container в фоне

-p 127.0.0.1:8080:80 публикует port только на loopback host

curl http://127.0.0.1:8080/ проверяет приложение

Если curl не отвечает, проверяют docker ps, docker port и docker logs

Port Publishing связывает host port и container port

127.0.0.1:8080:80

127.0.0.1 → host address

8080 → host port

80 → container port

docker port web показывает опубликованные порты

ss -lntp | grep 8080 показывает, слушается ли порт на Host

Публикация порта может открыть сервис наружу

-р 8080:80 может открыть сервис на внешних интерфейсах Host

-р 127.0.0.1:8080:80 ограничивает доступ loopback-адресом Host

Для первой лаборатории безопаснее loopback-публикация

Если нужен внешний доступ, нужно понимать firewall и listen address

Port Publishing не равен EXPOSE

EXPOSE и -p решают разные задачи

EXPOSE 80 документирует ожидаемый порт приложения

EXPOSE сам по себе не публикует порт на Host

-p 8080:80 реально публикует container port наружу

В Dockerfile EXPOSE помогает читать назначение image

При запуске доступность определяется docker run или Compose ports

Environment Variables подходят для настроек, но не для секретов

```
docker run -e APP_ENV=lab ...
```

Environment variable удобно передает режим работы приложения

Пароли и tokens лучше не хранить прямо в command line

Пароли и tokens лучше не хранить прямо в compose.yaml

Secret Manager и Docker secrets не разбираем глубоко в основной части

4. Storage Model разделяет временное и постоянное состояние

Writable Layer → временное состояние Container

Named Volume → Docker-managed persistent data

Bind Mount → конкретный каталог Host

Anonymous Volume можно знать обзорно

Для важных данных выбирают volume или bind mount,
а не Writable Layer

Named Volume переживает удаление container

`docker volume create webdata`

`docker volume ls`

`docker volume inspect webdata`

container удалили → named volume остался

`docker volume rm webdata` удаляет volume и данные

Volume дает persistence, но не заменяет backup

Volume не является Backup

Volume → persistence

Backup → recovery

Если volume удалили, данные потеряны

Если приложение испортило данные внутри volume, volume сохранит испорченное состояние

Для базы данных нужен отдельный backup-процесс

DB consistency подробно не разбирается в этой Docker-лекции

Bind Mount подключает конкретный каталог Host

```
mkdir -p html
```

```
echo 'Hello from bind mount' > html/index.html
```

```
docker run --name web -d -p 127.0.0.1:8080:80 --mount  
type=bind,source="$PWD/html",target=/usr/share/nginx/  
html,readonly nginx:stable-alpine
```

Host directory прямо виден Container

readonly не дает Container изменять эти данные

Bind Mount удобен для учебного HTML-каталога

UID и GID важны при Permission denied

Permission denied на Bind Mount требует проверки прав

`docker exec web id` показывает UID/GID процесса внутри container

`ls -ln html/` показывает владельца и права Host directory

Проблема может быть на стороне Host permissions
User namespaces в этой лекции не разбираем
подробно

Docker Network связывает containers по имени

`docker network create appnet`

User-defined network дает DNS по имени container

web может обращаться к db по имени db

Контейнеры в одной сети могут обмениваться

трафиком по внутренним портам

Bridge internals не раскрываем в первой лекции по Docker

Database port обычно не публикуют наружу

web → db:5432 внутри Docker Network

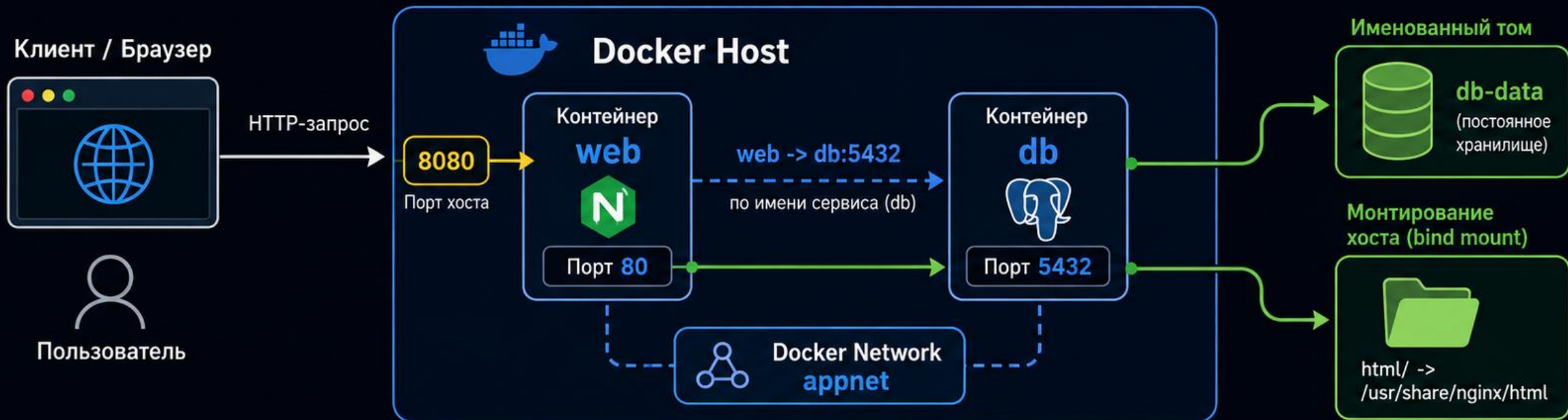
Если Database нужна только web, порт 5432 не публикуют на Host

Отсутствие ports у db не мешает web обращаться к db внутри appnet

Публикация database port наружу увеличивает поверхность атаки

Для junior важно различать internal container traffic и published port

Ports, Volumes и Docker Network



Ports

Host: **8080** -> web: **80**

Публикуем порт 8080 на хосте, который перенаправляется на порт 80 контейнера web.



Docker Network

web -> db:5432

Контейнеры в одной сети (appnet) видят друг друга по имени сервиса. Порт базы данных не публикуется наружу.



Volumes

db-data

Данные базы сохраняются в именованном томе. Файлы веб-приложения подключены через bind mount.



Главный вывод

Ports открывают доступ с хоста, Volumes сохраняют данные, Docker Network соединяет контейнеры между собой.

5. docker logs показывает stdout и stderr container

`docker logs web` - показать logs container

`docker logs --tail 100 web` - последние 100 строк

`docker logs --since 10m web` - logs за последние 10 минут

stdout/stderr container являются основным источником runtime logs

Logging drivers перечисляем только обзорно

Логи container тоже занимают disk host

Логи Docker могут заполнить диск Host

Log Rotation ограничивает размер и количество файлов

Compose-пример: driver: json-file

Compose options: max-size: "10m"

Compose options: max-file: "3"

Подробный logging pipeline не является темой этой лекции

docker exec нужен для диагностики, а не ручной настройки

`docker exec -it web sh`

exec открывает команду внутри работающего container

Через exec удобно проверить файл, процесс, DNS или доступность

Постоянную настройку нужно делать через Dockerfile или Compose

Не редактировать production container вручную через exec как способ настройки

Restart Policy реагирует на остановку главного процесса

no - не перезапускать автоматически

on-failure - перезапускать при ошибке процесса

always - запускать почти всегда

unless-stopped - запускать, если container не
остановлен администратором

Основной вариант лаборатории: unless-stopped

Restart Policy не исправляет любой сбой приложения

Healthcheck отличает Running от Healthy

Running $\neq$ Healthy

Container может работать, но приложение внутри не отвечать

Пример: HEALTHCHECK CMD wget -q --spider

http://localhost/ || exit 1

docker inspect web показывает информацию о healthcheck

Healthcheck в этой лекции базовый, без orchestration-подробностей

6. Dockerfile описывает сборку собственного image

FROM - базовый image

WORKDIR - рабочий каталог

COPY - копирование файлов в image

EXPOSE - документирование порта

CMD / ENTRYPOINT - команда запуска

USER - пользователь процесса внутри container

Простой Dockerfile для учебного Nginx

```
FROM nginx:stable-alpine
```

```
COPY index.html /usr/share/nginx/html/index.html
```

```
EXPOSE 80
```

Это минимальный понятный image для лаборатории

Healthcheck можно добавить вторым этапом

Не помещаем сразу весь production-hardening

Build превращает Dockerfile в Image

`docker build -t lab-nginx:1.0 .`

`docker image ls` - проверить появление image

Dockerfile → build → Image → run → Container

Tag `lab-nginx:1.0` нужен для запуска своего image

Правильный порядок инструкций ускоряет повторные builds

Если build не работает, проверяют Dockerfile и build context

.dockerignore защищает Build Context от лишнего

.git

*.log

*.pcap

*.bak

.env

secrets/

Секреты не должны попадать в Build Context

Первый Compose должен быть простым

```
services:
```

```
  web:
```

```
    image: lab-nginx:1.0
```

```
    ports:
```

```
      - "127.0.0.1:8080:80"
```

```
    restart: unless-stopped
```

Сначала студент должен понять один сервис web

Базовые команды Docker Compose

`docker compose config` - проверить итоговую конфигурацию

`docker compose up -d` - запустить проект в фоне

`docker compose ps` - показать containers проекта

`docker compose logs web` - посмотреть logs сервиса web

`docker compose restart` - перезапустить сервисы

`docker compose down` - остановить и удалить containers проекта

Compose + Bind Mount добавляем после базового запуска

volumes:

- `"./html:/usr/share/nginx/html:ro"`

Bind Mount подключает каталог `./html` внутрь Nginx
:`ro` делает mount read-only

Не добавляем сразу `read_only`, `tmpfs`, `security_opt`,
`logging` и `healthcheck` в первый пример

Улучшенный Compose показываем отдельным этапом

healthcheck - проверка состояния приложения

logging - ограничение logs

read_only - read-only root filesystem

tmpfs - временные writable-каталоги в памяти

no-new-privileges - запрет повышения привилегий

Junior не обязан применять все это в первой конфигурации

docker compose down -v опасен для данных

docker compose down → named volumes сохраняются

docker compose down -v → volumes проекта удаляются

Удаление volumes может привести к потере данных

Перед down -v нужно понять, где хранятся данные приложения

Для учебной работы студент должен уметь объяснить этот риск

depends_on не всегда означает готовность приложения

Короткий `depends_on` → порядок запуска `containers`
`healthcheck + condition: service_healthy` → можно
ожидать готовность `dependency`

Даже с `healthcheck` приложению полезна `retry logic`
`depends_on` не заменяет диагностику приложения

Для `junior` важно не путать запуск `container` и
готовность сервиса

7. Базовая безопасность Docker без перегруза hardening

Не использовать --privileged без необходимости

Не подключать Docker Socket обычному приложению

Не подключать host / целиком

Не давать лишние write mounts

По возможности запускать приложение без root

Обновлять Docker Host и используемые images

Docker Socket нельзя подключать как обычный volume

`-v /var/run/docker.sock:/var/run/docker.sock`

Container получает управление Docker Daemon

Такой container может влиять на Host

Это один из ключевых security warnings Docker

Подключать socket можно только при четком понимании риска

Advanced Hardening переносим в дополнительные темы

cap-drop и cap-add

seccomp, AppArmor и SELinux

Rootless Docker и userns-remap

image signing

Сначала нужен уверенный базовый жизненный цикл

Docker-приложения

Image Trust для junior начинается с простых правил

Использовать известный registry

Использовать известный image

Фиксировать версию image

Обновлять image и Docker Host

Digest можно использовать для строгой фиксации
содержимого

Vulnerability scanning и signing - дополнительные темы

Container не должен бесконтрольно забирать ресурсы Host

```
docker run --memory 256m --cpus 0.5 nginx:stable-alpine
```

--memory 256m ограничивает память container

--cpus 0.5 ограничивает CPU

Container ≠ неограниченные ресурсы Host

Resource limits особенно важны на сервере с несколькими containers

Prune не является универсальным лечением disk full

docker system prune может удалить нужные объекты

Сначала посмотреть: `docker system df`

Потом проверить: `docker ps -a`

Потом проверить: `docker image ls`

Потом проверить: `docker volume ls`

Только после понимания удалять лишнее

Единый практический сценарий: run и diagnose

1. docker version
2. docker pull nginx:stable-alpine
3. docker run
4. docker ps
5. docker port
6. curl
7. docker logs
8. docker exec

Единый практический сценарий: storage, build и compose

9. `docker stop/start`
10. Подключить read-only bind mount
11. Удалить container и убедиться, что данные Host сохранились
12. Создать Dockerfile
13. Выполнить `docker build`
14. Запустить собственный image
15. Создать `compose.yaml`
16. Выполнить `docker compose config` и `docker compose up -d`

Единый практический сценарий: приемка

17. Проверить приложение через curl
18. Посмотреть docker compose ps
19. Посмотреть docker compose logs web
20. Выполнить docker compose down
21. Объяснить риск docker compose down -v
22. Показать, где находятся данные: bind mount, volume или writable layer

Break/Fix: Port conflict и Container Exited

Port conflict: ошибка port already allocated

Проверка Docker: `docker ps`

Проверка Host: `ss -ltnp`

Container Exited: проверить `docker ps -a`

Дальше читать `docker logs web` и искать причину завершения главного процесса

Исправление: освободить port, выбрать другой host port или исправить команду запуска

Break/Fix: Bind Mount Permission и Service unavailable

Bind Mount Permission: проверить процесс командой
`docker exec web id`

Проверить права Host directory: `ls -ln html/`

Service unavailable: Container Running еще не
доказывает доступность приложения

Проверить Port: `docker port web` и `ss -ln tcp | grep 8080`

Проверить Curl: `curl http://127.0.0.1:8080/`

Проверить Logs и Application: `docker logs web`, файл,
конфигурация или healthcheck

Итоги лекции

Container использует Kernel Host, а VM имеет отдельную Guest OS и Kernel

Image является шаблоном, Container - экземпляром image
run, start, stop, restart и rm отвечают за разные этапы жизненного цикла

Данные нужно хранить в bind mount или volume, а не только в Writable Layer

Dockerfile создает image, Compose описывает повторяемый запуск

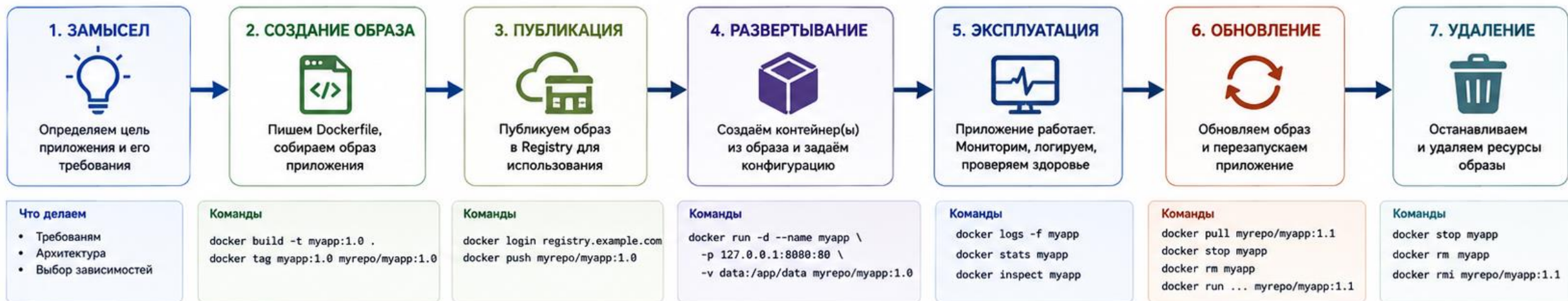
Junior должен уметь развернуть, проверить и диагностировать простое Docker-приложение

Контрольные вопросы

1. Чем Container отличается от VM?
2. Чем Image отличается от Container?
3. Чем `docker run` отличается от `docker start`?
4. Для чего используется `-p`?
5. Чем Bind Mount отличается от Named Volume?
6. Почему важные данные не хранят только в Writable Layer?
7. Для чего используются `docker logs` и `docker exec`?
8. Для чего нужен Dockerfile?
9. Для чего нужен Docker Compose?
10. Почему `docker compose down -v` опасен?
11. Почему опасно подключать Docker Socket в Container?

Диаграмма: жизненный цикл Docker-приложения

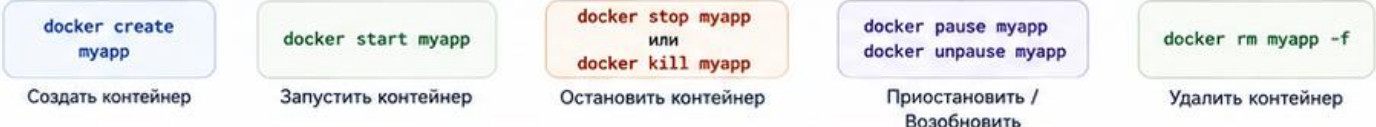
Путь от идеи до рабочего сервиса и обратно



СОСТОЯНИЯ КОНТЕЙНЕРА



Основные операции



ПОДКЛЮЧАЕМЫЕ РЕСУРСЫ



АРТЕФАКТЫ ДОКЕР-ПРИЛОЖЕНИЯ



Проверка и диагностика

- `docker ps -a` – список контейнеров
- `docker logs` – просмотр логов
- `docker exec -it myapp sh` – команды внутри
- `docker inspect myapp` – детальная информация
- `docker top myapp` – процессы в контейнере



Полезные практики

- Используйте образы с фиксированными тегами или digest
- Минимизируйте образ (alpine, multi-stage build)
- Не запускайте от root, задавайте пользователя
- Ограничивайте ресурсы (`--cpus`, `--memory`)
- Настройте healthcheck и restart policy



Пример healthcheck в Dockerfile

```
HEALTHCHECK --interval=30s --timeout=5s \
--retries=3 CMD curl -f http://localhost/health \
|| exit 1
```

Обозначения

- Переход к следующему шагу
- - - - - Возможный возврат к предыдущему шагу



Главная идея

Контейнеры – это неизменяемые артефакты. Обновляйте через создание новых образов и замену контейнеров.

Схема: ports, volumes и user-defined network

Как контейнеры получают доступ извне, сохраняют данные и общаются между собой



PORTS – доступ к контейнеру извне

Публикация порта хоста к порту контейнера



Пример команды

```
docker run -d \
  --name web \
  -p 127.0.0.1:8080:80 \
  nginx:stable-alpine
```

`-p <host_ip:host_port>:<container_port>`

- 127.0.0.1:8080 – доступ только с хоста
- Внешний порт 8080 проброшен во внутрь контейнера на 80/tcp
- Без `-p` сервис внутри контейнера недоступен извне

Проверка

```
> curl http://127.0.0.1:8080
```

Откройте в браузере <http://127.0.0.1:8080>



Полезно знать

- По умолчанию контейнеры изолированы и недоступны извне
- Публикуйте только нужные порты
- Привязывайте к 127.0.0.1 для локального доступа
- Используйте firewall/ufw на хосте для дополнительной защиты



VOLUMES – сохранение и обмен данными

Данные живут вне контейнера и не теряются при удалении



Варианты монтирования



Named Volume (рекомендуется для данных приложений)

```
docker run -d --name web \
  -p 127.0.0.1:8080:80 \
  -v webdata:/usr/share/nginx/html:ro \
  nginx:stable-alpine
```

Docker управляет расположением тома на хосте (обычно /var/lib/docker/volumes/...)



Bind Mount (для разработки и конфигов)

```
docker run -d --name web \
  -p 127.0.0.1:8080:80 \
  -v /opt/lab/html:/usr/share/nginx/html:ro \
  nginx:stable-alpine
```

Использует папку на хосте напрямую (изменения видны с обеих сторон)

Проверка

```
> docker volume ls
ls -la /opt/lab/html
```

Важно

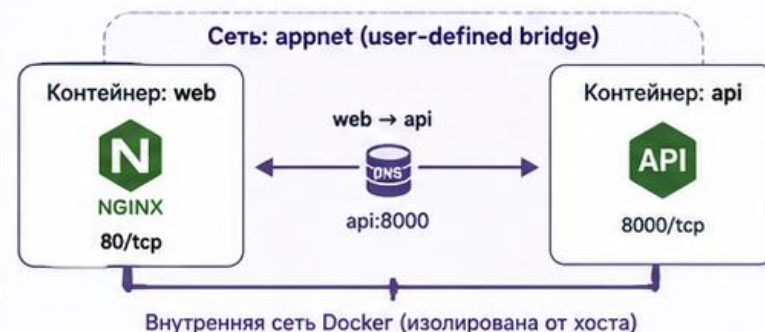


- Монтируйте только нужные пути
- Используйте `:ro` (read-only) где возможно
- Бэкапы делайте с томов, а не из контейнера



USER-DEFINED NETWORK – сеть для контейнеров

Контейнеры общаются по именам, а не по IP-адресам



Пример команд

1 Создаем сеть

```
docker network create appnet
```

2 Запускаем контейнеры в этой сети

```
docker run -d --name api --network appnet api:1.0
docker run -d --name web --network appnet -p 127.0.0.1:8080:80 nginx
```

3 Проверяем связь по имени

```
docker exec web wget -qO- http://api:8000/health
```

Преимущества user-defined сети



- Встроенный DNS: контейнеры обращаются по именам
- Изоляция: трафик не виден другим сетям по умолчанию
- Удобное управление и подключение к нужным сетям
- Поддержка драйверов (bridge, overlay и др.)

Итог: три ключевых механизма для практических сценариев



Ports – даем доступ извне

Публикуем только нужные порты и ограничиваем доступ (например, 127.0.0.1 или через firewall).



Volumes – сохраняем данные

Данные живут вне контейнера и не теряются. Используем named volumes или bind mounts.



User-defined network – соединяем сервисы

Контейнеры общаются по именам в изолированной сети. Просто, надежно и безопасно.

Команды на каждый день

```
> docker ps
docker logs -f <container>
docker inspect <container/network/volume>
docker rm -f <container>
```